

# P1705R1

## Enumerating Core Undefined Behavior

Draft Proposal, 2019-09-28

**This version:**

<http://wg21.link/P1705>

**Author:**

[Shafik Yaghmour](#) (Apple)

**Audience:**

SG12

**Toggle Diffs:**

☐ Hide deleted text

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

## Abstract

Adding an undefined behavior annex to the Standard and creating an C++ undefined behavior TR

## Table of Contents

|          |                                                   |
|----------|---------------------------------------------------|
| <b>1</b> | <b>Introduction</b>                               |
| <b>2</b> | <b>Edit History</b>                               |
| 2.1      | r0 → r1                                           |
| <b>3</b> | <b>Goals of Undefined Behavior Annex</b>          |
| <b>4</b> | <b>Benefits of Undefined Behavior Annex</b>       |
| <b>5</b> | <b>Implementation of Undefined Behavior Annex</b> |

|           |                                                                          |
|-----------|--------------------------------------------------------------------------|
| <b>6</b>  | <b>Stable Names</b>                                                      |
| <b>7</b>  | <b>Goals of Undefined Behavior TR</b>                                    |
| <b>8</b>  | <b>Benefits of Undefined Behavior TR</b>                                 |
| <b>9</b>  | <b>How would the Undefined Behavior TR relate to the Core Guidelines</b> |
| <b>10</b> | <b>What about Standard Library Undefined Behavior?</b>                   |
| <b>11</b> | <b>How Might the TR look</b>                                             |
| 11.1      | [lex]                                                                    |
| 11.1.1    | [lex.phases]                                                             |
| 11.1.2    | [lex.string]                                                             |
| 11.2      | [basic]                                                                  |
| 11.2.1    | [basic.def.odr]                                                          |
| 11.2.2    | [basic.life]                                                             |
| 11.2.3    | [basic.indet]                                                            |
| 11.2.4    | [basic.start]                                                            |
| 11.3      | [expr]                                                                   |
| 11.3.1    | [expr.pre]                                                               |
| 11.3.2    | [conv.double]                                                            |
| 11.3.3    | [conv.fpoint]                                                            |
| 11.3.4    | [expr.call]                                                              |
| 11.3.5    | [expr.static.cast]                                                       |
| 11.3.6    | [expr.delete]                                                            |
| 11.3.7    | [expr.mptr.oper]                                                         |
| 11.3.8    | [expr.mul]                                                               |
| 11.3.9    | [expr.add]                                                               |
| 11.3.10   | [expr.shift]                                                             |
| 11.3.11   | [expr.ass]                                                               |
| 11.4      | [stmt.stmt]                                                              |
| 11.4.1    | [stmt.return]                                                            |
| 11.4.2    | [stmt.dcl]                                                               |
| 11.5      | [dcl.dcl]                                                                |
| 11.5.1    | [dcl.type.cv]                                                            |

|        |                           |
|--------|---------------------------|
| 11.5.2 | [dcl.attr.contract.syn]   |
| 11.5.3 | [dcl.attr.contract.syn]   |
| 11.5.4 | [dcl.attr.contract.check] |
| 11.5.5 | [dcl.attr.noreturn]       |
| 11.6   | [class]                   |
| 11.6.1 | [class.mfct.non-static]   |
| 11.6.2 | [class.dtor]              |
| 11.6.3 | [class.union]             |
| 11.6.4 | [class.abstract]          |
| 11.6.5 | [class.base.init]         |
| 11.6.6 | [class.cdtor]             |

## 12 Acknowledgement

### References

Informative References

## § 1. Introduction

Explaining undefined behavior is complicated. First you need to explain what undefined behavior is. Then all the unintuitive consequences that it entails. Including removal of safety checks, turning finite loops infinite, booleans that can both be false and true and how undefined behavior can time travel 🤖

Then comes the next logical question, how can I know what all the undefined behavior are so I can avoid them. This may be followed by an awkward silence, “That is complicated”, one might say. We might follow up and mention we have both explicit and implicit undefined behavior. A fair response might be, “Makes sense but surely you can tell me what all the explicit undefined behaviors are?”. This would be followed by more awkward silence. Followed by a perhaps sheepish, “Well you see that is also complicated”.

We would have to follow-up and point out that the C++ Standard does indeed list all the explicit undefined behavior but you would have to manually go through the 1700+ page Standard to find them. Merely finding all the mentions of “undefined” is only partially helpful. The Standard being a specification and not a tutorial does not explain each in plain language and honestly some defy explanation in plain language. Examples are not always provided, neither do we have rationales or explanations on how to avoid or catch violations of these rules (if possible).

The goal of this paper is two fold. One is to create an annex of undefined behavior. The purpose would be to have a list of all the explicit core undefined behavior along with at least one example demonstrating it. Having this list will enable the C++ community to better grasp the scope and depth of undefined behavior. It should benefit not just users but also those teaching C++ and those developing tools for writing better code. It will benefit implementors because it lets them know what's undefined and how. It will help the committee track its undefined behavior and revisit it.

The second goal would be to develop of a core undefined behavior TR, which would expand upon the content of the annex with more examples including examples showing surprising consequences. It would also include tools if any that could aid in detecting or avoiding each undefined behavior. If possible we would also like to include a rationale for each undefined behavior. This will have all the benefits of that annex but with more details and rationale should aid in teaching. Additionally this should also be a help to researchers both in understanding, developing better tools and perhaps finding alternatives approaches to undefined behavior.

## § 2. Edit History

### § 2.1. r0 → r1

[\[P1705R0\]](#) was seen by SG12 in Cologne. This update does the following:

- Update the implementation of undefined behavior annex section
- Add details on how implicit undefined behavior will be handled in the annex
- Add details on stable names for each undefined behavior entry
- Add an example of how the annex will be formatted

| Poll                                                                                                                                                                                                                   | Group | SF | F | N | A | SA | Outcome |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|----|---|---|---|----|---------|
| <b>Add an informative annex to the WD which lists core language UB organized by type to be maintained by someone after initial version from author.</b>                                                                | SG12  | 8  | 2 | 0 | 0 | 0  | ✓       |
| <b>Include implicit (by omission) UB on a best effort basis.</b>                                                                                                                                                       | SG12  | 5  | 4 | 1 | 0 | 0  | ✓       |
| <b>Target an informative document (TR/SD/?) that contains UB along with rationale and additional information to be maintained by SG12 after initial version from author along the lines of the contents of P1705R0</b> | SG12  | 3  | 3 | 3 | 1 | 0  | ✓       |

### § 3. Goals of Undefined Behavior Annex

- List all the explicit undefined behavior in the core language.
- List implicit undefined behavior in the core language on a best effort basis.
- Provide at least one example for each undefined behavior.

### § 4. Benefits of Undefined Behavior Annex

- A definitive list of all explicit core language undefined behavior.
- Ability to track the velocity of undefined behavior in the Standard. Is it increasing/decreasing and at what rate.
- Allow of better analysis:
- Are there classes of undefined behavior we can get better at detecting?
- Are there classes of UB that are candidates to convert to something else such as ill-formed or implementation defined behavior.
- Provide clarity with examples.

### § 5. Implementation of Undefined Behavior Annex

Annex E will be modeled similar Annex C with a description of each undefined behavior and at least one example. Each Annex E entry will have its own stable name that we can use to refer to the specific undefined behavior unambiguously.

There will be two LaTeX macros: one to link each annex entry back to the normative wording via a reference, and a second macro to link the normative wording to the Annex E via a reference. Having a two way reference will us to maintain the Annex E more seamlessly without having to worry about the normative text moving or being removed without this being reflected in Annex E.

### § 6. Stable Names

Stables names will take the form of `ub.SectionName.UniqueName`. For example:

- `[ub.lex.phases.spliceucn]`
- `[ub.lex.phases.tokenconcatucn]`

- [ub.lex.string.modstringliteral]
- [ub.basic.def.odr]

## § 7. Goals of Undefined Behavior TR

- It is easier to adapt the Standard because we understand the decisions that were made and can revisit them.
- Provide examples that demonstrate surprising consequences.
- Provide a rationale for each undefined behavior.
- Provide ways to prevent the undefined behavior.
- Provide ways to detect the undefined behavior.
- Provide live example on Compiler Explorer or other live tools that demonstrate the undefined behavior. If it can demonstrate ways to detect the undefined behavior then also include that.

## § 8. Benefits of Undefined Behavior TR

- Ease teaching by providing a comprehensive reference.
- Ease tool development.
- Ease research into undefined behavior.

## § 9. How would the Undefined Behavior TR relate to the Core Guidelines

The C++ Core Guidelines are focused on "relatively high-level issues", which is appropriate for a document that seeks to "help people to use modern C++ effectively". Undefined behavior itself may be one high-level topic and does deserve specific mention in the Core Guidelines. The Undefined Behavior TR would be more focused, drilling into each specific core undefined behavior with details. The undefined behavior TR would therefore inform the Core Guidelines.

## § 10. What about Standard Library Undefined Behavior?

Undefined behavior is a large topic, to make it a more tractable problem we believe tackling Core undefined behavior separately from Library undefined behavior makes sense. Core and Library already have separate processes and tackling them separately will allow those with expertise in Core or

Library to focus on those areas repsectively. This proposal specifically focuses on Core while acknowledging that documenting Library undefined behavior is important, we leave that to a future proposal.

## § 11. How Might the TR look

There has been some effort to document core undefined behavior and below I will provide an example of one approach to an undefined behavior TR. This works covers about most of the explicit core undefined behavior with at least one example for each undefined behavior. To a lesser extent it covers rationales, backgrounds and tools:

### § 11.1. [lex]

#### § 11.1.1. [lex.phases]

- if a splice results in a character sequence that matches the syntax of a universal-character-name, the behavior is undefined.
- [lex.phases p1.2](#)
- Example from [Stack Overflow question](#):

```
const char* p = "\\
u0041";
```

- [Examples live](#)
- Rationale
  - [DR 787](#)
  - [N3881 “Fixing the specification of universal-character-names”](#)
- If a character sequence that matches the syntax of a universal-character-name is produced by token concatenation (19.3.3), the behavior is undefined. *\[lex.phases\]/p4*
- [lex.phases p1.4](#)
- Examples:

```
#define GUARD_NAME i ## _GUARD // UB per current spec
#define COLUMN "i" ## _column // UB per current spec
```

- [Examples live](#)
- Rationale
  - [DR 787](#)
  - [N3881 “Fixing the specification of universal-character-names”](#)

### § 11.1.2. [lex.string]

- The effect of attempting to modify a string literal is undefined
- Examples:

```
const char *p1 = "hello world\n";
char *p2 = const_cast<char*>(p1) ;    // const_cast is already suspicious

p2[0] = 'm' ;
```

- [Examples live](#)
- Rationale [lex.string]p8

Ordinary string literals and UTF-8 string literals are also referred to as narrow string literals. A narrow string literal **has type “array of n const char”**, where n is the size of the string as defined below, and has static storage duration (6.6.4).

## § 11.2. [basic]

### § 11.2.1. [basic.def.odr]

- There can be more than one definition of a class type (Clause 12), enumeration type (10.2), inline function with external linkage (10.1.6), inline variable with external linkage (10.1.6), class template (Clause 17), non-static function template (17.6.6), concept (17.6.8), static data member of a class template (17.6.1.3), member function of a class template (17.6.1.1), or template specialization for which some template parameters are not specified (17.8, 17.6.5) in a program provided that each definition appears in a different translation unit, and provided the definitions satisfy the following requirements. Given such an entity named D defined in more than one translation unit, then ... If the definitions of D do not satisfy these requirements, then the behavior is undefined.

- [basic.def.odr p12](#)
- [Examples from: DCL60-CPP. Obey the one-definition rule](#)

```
// a.cpp
struct S {
    int a;
};

// b.cpp
// S in b.cpp does not consist of the same sequence of token as S i
class S {
public:
    int a;
};

const int n = 42;

int g(const int &lhs, const int &rhs);

inline int f(int k) {
    return g(k, n);    // f() has external linkage
                      // n has internal linkage but is our-used by g()
                      // n will not be identical in all trans
}
```

- Rationale:
  - [Devirtualization in C++, part 7 \(Enforcing One Definition Rule\)](#)

### § 11.2.2. [basic.life]

- A program may end the lifetime of any object by reusing the storage which the object occupies or by explicitly calling the destructor for an object of a class type with a non-trivial destructor. For an object of a class type with a non-trivial destructor, the program is not required to call the destructor explicitly before the storage which the object occupies is reused or released; however, **if there is no explicit call to the destructor or if a delete-expression ([expr.delete]) is not used to release the storage, the destructor shall not be implicitly called and any program that depends on the side effects produced by the destructor has undefined behavior.**

- [basic.life p5](#)
- [Pull request 2342 indicates this may not be undefined behavior at all](#) and seeks the following edit ~~implicitly called and any program that depends on the side effects produced by the destructor has undefined behavior~~implicitly called.

### § 11.2.3. [basic.indet]

- If an indeterminate value is produced by an evaluation, the behavior is undefined except in the following cases
- [basic.indet p2](#)
- Examples

```
int f(bool b) {
    unsigned char c;
    unsigned char d = c; // OK, d has an indeterminate value
    int e = d; // undefined behavior
    return b ? d : 0; // undefined behavior if b is true
}
```

- Rationale
  - [WG14 Defect report 260](#)
  - [WG14 Defect report 451](#)
  - Tl;DR; We have two case one in which using an indeterminate value is undefined behavior and this is because many type can have trap representations and using these value are undefined behavior. In the case of narrow character types the underlying values and type representation are one to one and therefore we don't have a trap representation but they do retain their indeterminateness.

### § 11.2.4. [basic.start]

- An implementation shall not predefine the main function. This function shall not be overloaded. Its type shall have C++ language linkage and it shall have a declared return type of type int, but otherwise its type is implementation-defined.
  - [basic.start.main p2](#)
  - Examples:

```
void main() {}
```

- Tools
- [Compiler static analysis, generates warnings or errors for void main](#)
- The function main shall not be used within a program
  - [basic.start.main p3](#)
  - Examples:
  - [Is it illegal to take address of main\(\) function?](#)
  - [Why does gcc warn about decltype\(main\(\)\) but not clang?](#)

```
printf( "%p\n", &main ) ;  
decltype(main()) x = 0;
```

```
int main() {  
    std::cout << reinterpret_cast<void*>(&main) ;  
}
```

- Tools
  - [Compiler static analysis via -pedantic flag gcc/clang for taking address of main](#)
  - [Compiler static analysis via -pedantic flag gcc/clang decltype main](#)
- Rationale:
  - From ARM section 3.4 Start and Termination

This is to ensure full freedom of the implementation of the interface between a C++ program and its environment. One could imagine an implementation where main() was not implemented as a function.
  - [What does "The function main shall not be used within a program" mean?](#)

## § 11.3. [expr]

### § 11.3.1. [expr.pre]

- Signed integer overflow/underflow is undefined behavior

- [expr.pre](#)
- Examples

```
int x1=std::numeric_limits<int>::max()+1;
int x2=std::numeric_limits<int>::min()-1;
int x3=std::numeric_limits<int>::min() / -1;
```

- [Examples live 1](#) and [examples live 2](#)

### § 11.3.2. [conv.double]

- Converting floating point value to type that cannot represent the value is undefined behavior even for float
- [conv.double p1](#)
- Examples

```
double d2=DBL_MAX;
float f=d2;
```

- [examples live](#)

### § 11.3.3. [conv.fpint]

- Converting floating point value to an integral that cannot represent the value is undefined behavior
- [conv.fpint p1](#)
- Examples

```
double d=(double)INT_MAX+1;
int x=d;
```

- [Examples live](#)

#### § 11.3.4. [expr.call]

- Calling a function through an expression whose function type is different from the function type of the called function's definition results in undefined behavior
- [expr.call p6](#)
- Examples

```
int f_c(int);

using c1 = int (*)(int);
using c2 = int (*)(int,int);

int f(c2 func) {
    return func(1,2);
}

int main() {
    f(reinterpret_cast<c2>(f_c));
}
```

- [Examples live](#)

#### § 11.3.5. [expr.static.cast]

- If the object of type “cv1 B” is actually a base class subobject of an object of type D, the result refers to the enclosing object of type D. Otherwise, the behavior is undefined.
- [expr.static.cast](#)
- Examples:

```
struct B {};
struct D1:B {};
struct D2:B {};

void f() {
    D1 d;
    B &b = d;
    static_cast<D2&>(b);
}
```

- [Examples live](#)
- Setting an enum to a value outside the range of enumerators is undefined behavior
- [expr.static.cast p10](#) and [\[dcl.enum\\]p8](http://eel.is/c++draft/dcl.enum#8)(<http://eel.is/c++draft/dcl.enum#8>)
- Examples

```
enum A {e1=1, e2};

void f() {
    enum A a=static_cast<A>(4);
}
```

- [Examples live](#)
- Down-casting to the wrong derived type is undefined behavior
- [expr.static.cast p11](#)
- Examples

```
struct B {};
struct D1:B {};
struct D2:B {};

void f() {
    B* bp = new D1;
    static_cast<D2*>(bp);
}
```

- [Examples lives](#)

### § 11.3.6. [expr.delete]

- Using array delete on the result of a single object new expression and vice versa is undefined behavior
- [\[expr.delete p2\]](http://eel.is/c++draft/expr.delete#2)(<http://eel.is/c++draft/expr.delete#2>)
- Examples

```
int *x = new int;
delete [] x;
```

- [Examples live](#)
- If the dynamic type differs from the static type of the object being deleted that is undefined behavior
- [expr.delete p3](#)
- Examples

```
int *p = new int;
float *f = reinterpret_cast<float*>(p);
delete f;
```

- [Examples lives](#)
- Deleting and incomplete type and the class turns out to have a non-trivial destructor is undefined behavior
- [expr.delete p5](#)
- Examples

```
struct A;

void f(A *p) {
    delete p;
}

struct A {~A() {}};
```

- [Examples live](#)

### § 11.3.7. [expr.mptr.oper]

- If the dynamic type of E1 does not contain the member to which E2 refers, the behavior is undefined
- [expr.mptr.oper p4](#)
- Examples:

```

struct B{};
struct D:B{int x;};

void f(){
    B *b= new B;
    D *d=static_cast<D*>(b);
    int D::* p=&D::x;
    (*d).*p=1;
}

```

- [Examples live](#)
- If the second operand is the null member pointer value (7.3.12), the behavior is undefined.
- [expr.mptr.oper p6](#)
- Examples:

```

struct S {
    int i;
};

void f()
{
    S cs;
    int S::* pm = nullptr;
    cs.*pm = 88;
}

```

- [Examples live](#)

### § 11.3.8. [expr.mul]

- Division by zero is undefined behavior
- [expr.mul p4](#)
- Examples:

```

int x = 1/0;
double d = 1.0/0.0;

```

- [Examples live](#)

### § 11.3.9. [expr.add]

- Incrementing pointer beyond one past the end of an array is undefined behavior
- [expr.add p4](#) and [footnote](#)
- Examples:

```
static const int arrs[10]{};

void f() {
    const int* y = arrs + 11;
}
```

- [Examples live](#)
- Subtracting pointers that are not part of the same array is undefined behavior
- [expr.add p5.3](#)
- Examples:

```
void f() {
    int x;
    int y;
    int *p1=&x;
    int *p2=&y;
    std::ptrdiff_t off = p1-p2;
}
```

- [Examples live](#)

### § 11.3.10. [expr.shift]

- Shifting by a negative amount is undefined behavior
- [expr.shift p1](#)
- Examples

```
int y = 1 << -1;
```

- [Examples live](#)

- Shifting by equal or greater than the bit-width of a type is undefined behavior
- [expr.shift p1](#)
- Examples:

```
int y1 = 1 << 32;
int y2 = 1 >> 32;
```

- [Examples live](#)
- Shifting a negative signed type is undefined behavior (before C++20)
- [expr.shift p2](#)
- Examples:

```
int y4 = -1 << 12;
```

- [Examples live](#)

#### § 11.3.11. [expr.ass]

- Overlap in an assignment expression must be exact and the objects must have the same type
- [expr.ass p8](#)
- Examples:

```
int x=1;
char *c=reinterpret_cast<char*>(&x);
x = *c;
```

- [Examples live](#)

### § 11.4. [stmt.stmt]

#### § 11.4.1. [stmt.return]

- Flowing off the end of a value returning function is undefined behavior
- [stmt.return p2](#)
- Examples:

```

int f(int x) {
    if(x)
        return 1;
}

void b(){
    int x=f(0);
}

```

- [Examples live](#)
- [Also see](#)

#### § 11.4.2. [stmt.dcl]

- Recursively entering declaration of a block scope static variable during initialization is undefined behavior
- [stmt.dcl p4](#)
- Examples:

```

int foo(int i) {
    static int s = foo(2*i);
    return i+1;
}

```

- [Examples live](#)

### § 11.5. [dcl.dcl]

#### § 11.5.1. [dcl.type.cv]

- Attempting to modify a const object is undefined behavior
- [dcl.type.cv p4](#)
- Examples:

```

int bar() {
    const int x=1;

    int *p = const_cast<int*>(&x);
    *p = 2;

    return *p;
}

```

- [Examples live](#)
- Accessing a volatile value through a non-volatile is undefined behavior
- [dcl.type.cv p5](#)
- Examples:

```

void f() {
    volatile int x=0;
    int &y=const_cast<int&>(x);
    std::cout << y;
}

```

[-Examples live](#)

### § 11.5.2. [dcl.attr.contract.syn]

- In contracts side effects in a predicate to an object whose lifetime did not begin and end within the evaluation of the predicate are undefined behavior
- [dcl.attr.contract.syn p6](#)
- Examples:

```

int min = -42;
constexpr int g(int x) {
    /* ... */
    [[assert: ++min > 0]]; // undefined behavior
    /* ... */
    return 0;
}

```

- [Examples](#)

```
live%7B%0A++/+...+/%0A++%5B%5Bassert:+%2B%2Bmin+%3E+0%5D%5D%3B+//+undefi
ned+behavior%0A++/+...+/%0A++return+0%3B%0A%7D'),l:'5',n:'0',o:'C%2B%2B+source+%
231',t:'0')),k:50,l:'4',n:'0',o:',s:0,t:'0'),(g:!((g:!((h:compiler,i:(compiler:clang%2B%2B-
master,filters:
(b:'0',binary:'1',commentOnly:'0',demangle:'0',directives:'0',execute:'1',intel:'0',trim:'0'),lang
:c%2B%2B,libs:!(),options:',source:1),l:'5',n:'0',o:'Clang+6.0.0+x86_64+%5Bclang-
contracts%5D+(master)+
(Editor+%231,+Compiler+%231)+C%2B%2B',t:'0')),k:50,l:'4',m:50,n:'0',o:',s:0,t:'0'),(g:!
((h:output,i:(compiler:1,editor:1),l:'5',n:'0',o:'%231+with+Clang+6.0.0+x86_64+%5Bclang-
contracts%5D+(master)',t:'0')),header:
(),l:'4',m:50,n:'0',o:',s:0,t:'0')),k:50,l:'3',n:'0',o:',t:'0')),l:'2',n:'0',o:',t:'0')),version:4)
```

### § 11.5.3. [dcl.attr.contract.syn]

- if a postcondition odr-uses a non-reference parameter in its predicate and the function body makes direct or indirect modifications of the value of that parameter, the behavior is undefined.
- [dcl.attr.contract.cond p7](#)
- Examples:

```
int f(int x)
[[ensures r: r == x]]
{
    return ++x; // UB
}
```

- [Examples](#)

```
live%0A++%5B%5Bensures+r:r+%3D%3D+x%5D%5D%0A%7B%0A++return+%2B%2Bx%
3B+//+UB%0A%7D%0A'),l:'5',n:'0',o:'C%2B%2B+source+%231',t:'0')),k:50,l:'4',n:'0',o:',s:0,
t:'0'),(g:!((g:!((h:compiler,i:(compiler:clang%2B%2B-master,filters:
(b:'0',binary:'1',commentOnly:'0',demangle:'0',directives:'0',execute:'1',intel:'0',trim:'0'),lang
:c%2B%2B,libs:!(),options:',source:1),l:'5',n:'0',o:'Clang+6.0.0+x86_64+%5Bclang-
contracts%5D+(master)+
(Editor+%231,+Compiler+%231)+C%2B%2B',t:'0')),k:50,l:'4',m:50,n:'0',o:',s:0,t:'0'),(g:!
((h:output,i:(compiler:1,editor:1),l:'5',n:'0',o:'%231+with+Clang+6.0.0+x86_64+%5Bclang-
contracts%5D+(master)',t:'0')),header:
(),l:'4',m:50,n:'0',o:',s:0,t:'0')),k:50,l:'3',n:'0',o:',t:'0')),l:'2',n:'0',o:',t:'0')),version:4)
```

#### § 11.5.4. [dcl.attr.contract.check]

- Violating a non-checked contract is undefined behavior outside of a constant expression context
- [dcl.attr.contract.check p4](#)
- Rationale see [p1321r0](#) and [p1490r0](#)
- Examples:

```
void f(int x) [[expects audit: x>=1 && x<=2]];

void b() {
    f(100);
}
```

- [Examples](#)  
[live](#)+%5B%5Bexpects+audit:+x%3E%3D1+%26%26+x%3C%3D2%5D%5D%3B%0A%0Aavoid+b()+%7B%0A++f(100)%3B%0A%7D'),l:'5',n:'0',o:'C%2B%2B+source+%231',t:'0')),k:50,l:'4',n:'0',o:',s:0,t:'0'),(g:!((h:compiler,i:(compiler:clang%2B%2B-master,filters:(b:'0',binary:'1',commentOnly:'0',demangle:'0',directives:'0',execute:'1',intel:'0',trim:'0'),lang:c%2B%2B,libs:!(),options:',source:1),l:'5',n:'0',o:'Clang+6.0.0+x86\_64+%5Bclang-contracts%5D+(master)+(Editor+%231,+Compiler+%231)+C%2B%2B',t:'0')),k:50,l:'4',m:50,n:'0',o:',s:0,t:'0'),(g:!((h:output,i:(compiler:1,editor:1),l:'5',n:'0',o:'%231+with+Clang+6.0.0+x86\_64+%5Bclang-contracts%5D+(master)',t:'0')),header:(),l:'4',m:50,n:'0',o:',s:0,t:'0')),k:50,l:'3',n:'0',o:',t:'0')),l:'2',n:'0',o:',t:'0')),version:4)

#### § 11.5.5. [dcl.attr.noreturn]

- A function declared noreturn eventually returns it is undefined behavior
- [dcl.attr.noreturn p2](#)
- Examples:

```
[[ noreturn ]] void q(int i) { // behavior is undefined if called with
if (i > 0)
    throw "positive";
}
```

- [Examples live](#)

## § 11.6. [class]

### § 11.6.1. [class.mfct.non-static]

- Calling a non-static member function of a class with an object that is not of that type is undefined behavior
- [class.mfct.non-static p2](#)
- Examples:

```
struct X {  
    int x=1;  
    int f() { return x;}  
};  
  
struct A {int x=3;};  
  
int f(X*x) {  
    return x->f();  
}
```

- [Examples live](#)

### § 11.6.2. [class.dtor]

- Explicit destructor call for an object not of the type is undefined behavior
- [class.dtor p14](#)
- Examples:

```
struct X {};  
  
void f() {  
    X *x=nullptr;  
    x->~X();  
}
```

- [Examples live](#)
- Invoking the destructor for an object once its lifetime has ended is undefined behavior
- [class.dtor p16](#)

- Examples:

```
struct A{
    ~A(){}
};

int main() {
    A a;
    a.~A(); // Destructor will be invoked again at scope exit invoking
}
```

- [Examples live](#)

### § 11.6.3. [class.union]

- Accessing a non-active union member is undefined behavior
- [class.union p1](#)
- Examples:

```
union Y { float f; int k; };
void g() {
    Y y = { 1.0f }; // OK, y.f is active union member (10.3)
    int n = y.k;
}
```

- [Examples live](#)

### § 11.6.4. [class.abstract]

- Calling a virtual function from a constructor or destructor in an abstract class is undefined behavior
- [class.abstract p6](#)
- Examples:

```

struct B {
    virtual void f()=0;
    B() { f();}
};

struct D : B{
    void f() override { }
};

```

- [Examples live](#)

#### § 11.6.5. [class.base.init]

- Calling a member function before all bases are initialized is undefined behavior
- [class.base.init p16](#)
- Examples:

```

struct B {
    B(int);
};

struct D : public B {
    int f();
    D() : B(f()) {}
};

```

- [Examples live](#)

#### § 11.6.6. [class.ctor]

- For an object with a non-trivial constructor, referring to any non-static member or base class of the object before the constructor begins execution results in undefined behavior
- [class.ctor p1](#)
- Examples

```

struct W { int j; };
struct X : public virtual W { };

struct Y {
    int* p;
    X x;
    Y() : p(&x.j) { // undefined, x is not yet constructed
    }
};

```

- [Examples live](#)
- To explicitly or implicitly convert a pointer (a glvalue) referring to an object of class X to a pointer (reference) to a direct or indirect base class B of X, the construction of X and the construction of all of its direct or indirect bases that directly or indirectly derive from B shall have started and the destruction of these classes shall not have completed, otherwise the conversion results in undefined behavior
- [class.ctor p3](#)
- Examples:

```

struct A { };
struct B : virtual A { };
struct C : B { };
struct D : virtual A { D(A*); };
struct X { X(A*); };

struct E : C, D, X {
    E() : D(this), // undefined: upcast from E* to A* might use path E*
              // but D is not constructed

              // "D((C*)this)" would be defined: E* ! C* is def
              // and C* ! A* is defined because C is fully const

    X(this) {} // defined: upon construction of X, C/B/D/A sublattice
};

```

- [Examples live](#)
- To form a pointer to (or access the value of) a direct non-static member of an object obj, the construction of obj shall have started and its destruction shall not have completed, otherwise the computation of the pointer value (or accessing the member value) results in undefined behavior.

- [class.cdtor p3](#)
- Examples:

```
struct A {  
    int x;  
};  
  
void f() {  
    A a;  
    a.~A();  
    int *p=&a.x; // Destruction completed so computing the pointer is u  
}
```

- [Examples lives](#)
- If the virtual function call uses an explicit class member access (7.6.1.4) and the object expression refers to the complete object of x or one of that object's base class subobjects but not x or one of its base class subobjects, the behavior is undefined.
- [class.cdtor p4](#)
- Examples:

```

struct V {
    virtual void f();
    virtual void g();
};

struct A : virtual V {
    virtual void f();
};

struct B : virtual V {
    virtual void g();
    B(V*, A*);
};

struct D : A, B {
    virtual void f();
    virtual void g();
    D() : B((A*)this, this) { }
};

B::B(V* v, A* a) {
    f(); // calls V::f, not A::f
    g(); // calls B::g, not D::g
    v->g(); // v is base of B, the call is well-defined, calls B::g
    a->f(); // undefined behavior, a's type not a base of B.
}

```

- [Examples live](#)
- If the operand of typeid refers to the object under construction or destruction and the static type of the operand is neither the constructor or destructor's class nor one of its bases, the behavior is undefined
- [class.ctor p5](#)
- Examples:

```

struct V {
    virtual void f();
};

struct A : virtual V { };
struct B : virtual V {
    B(V*, A*);
};

struct D : A, B {
    D() : B((A*)this, this) { }
};

B::B(V* v, A* a) {
    typeid(*this); // type_info for B.
    typeid(*v); // well-defined: *v has type V, a base of B yields type
    typeid(*a); // undefined behavior: type A not a base of B
    dynamic_cast<B*>(v); // well-defined: v of type V*, V base of B res
    dynamic_cast<B*>(a); // undefined behavior, a has type A*, A not a
}

```

- [Examples live](#)
- If the operand of the `dynamic_cast` refers to the object under construction or destruction and the static type of the operand is not a pointer to or object of the constructor or destructor's own class or one of its bases, the `dynamic_cast` results in undefined behavior
- [class.ctor p6](#)
- Examples:

```

    struct V {
        virtual void f();
    };

    struct A : virtual V { };
    struct B : virtual V {
        B(V*, A*);
    };

    struct D : A, B {
        D() : B((A*)this, this) { }
    };

    B::B(V* v, A* a) {
        typeid(*this); // type_info for B.
        typeid(*v); // well-defined: *v has type V, a base of B yields type
        typeid(*a); // undefined behavior: type A not a base of B
        dynamic_cast<B*>(v); // well-defined: v of type V*, V base of B res
        dynamic_cast<B*>(a); // undefined behavior, a has type A*, A not a
    }

```

- [Examples live](#)

## § 12. Acknowledgement

Thanks to JF Bastien for his review.

## § References

### § Informative References

#### [P1705R0]

Shafik Yaghmour. [Enumerating Core Undefined Behavior](#). 13 June 2019. URL:  
<https://wg21.link/p1705r0>